

Project II — Web Programming

2025/26 · Class 2426 · ETIC_Algarve

Project II — Web Programming	1
2025/26 · Class 2426 · ETIC Algarve	1
1. Context	2
2. Objectives	2
3. Projects & Groups	2
4. Phases & Sessions	3
Phase 1 — Scope Definition & Planning (Sessions 1–3)	4
Phase 2 — Design, Architecture & Data Modelling (Sessions 4–6)	4
Phase 3 — Development: Core MVP (Sessions 7–12)	4
Phase 4 — Refinement, Testing & Documentation (Sessions 13–19)	4
Phase 5 — Final Delivery & Presentation (Sessions 20–22)	5
5. Technical Requirements	5
5.1 Functional Requirements (FR)	5
5.2 Non-Functional Requirements (NFR)	5
5.3 Bonus (not mandatory)	6
6. Deliverables	6
7. Procedures & Git Workflow	6
8. Evaluation Criteria	7
8.1 Criterion breakdown	7
8.2 Individual vs. group score	8
9. Evaluation Methods	8
10. Plagiarism & AI Misuse	9
10.1 Allowed	9
10.2 Required	9
10.3 Prohibited	9
10.4 Consequences	9
11. Timeline & Key Dates	10
Late submission policy	10
12. Final Considerations	10

1. Context

Project II is the capstone module of the 2nd year of ETIC_Algarve's Web Programming course.

It is the point at which students consolidate two years of training into a single, deliverable product.

For the 2025/26 academic year, Class 2426 will be organised into **four groups**, each addressing a distinct brief:

- **One group** will work on a real-client project ("Integra-te") delivered by an external partner, the Fundação António Aleixo. This group is augmented with one student from Class 2527 (1st year) to provide additional capacity and cross-year collaboration.
- **Three groups** will each take a pitch previously developed in the *Pitching & Entrepreneurship* module and turn it into a functional **MVP (Minimum Viable Product)**.

All projects must be delivered by **26th June 2026 at 23h59m.**

2. Objectives

By the end of Project II, each group will have:

1. **Scoped and planned** a web-based product, balancing business goals and technical constraints.
 2. **Designed and implemented** a full-stack solution with a clear frontend, backend and data layer.
 3. **Containerised** the entire solution using Docker and Docker Compose.
 4. **Documented** the product so it can be installed, maintained and handed over.
 5. **Defended** the resulting work - both in code review and in a final presentation - with every team member able to explain any portion of the delivered product, regardless of who originally wrote it.
-

3. Projects & Groups

Group composition is set by the tutor. Project allocation is fixed for Group A (Integra-te) and selected from pitch projects for the remaining groups, subject to tutor approval in Week 1.

Group	Members	Project	Type	Brief
A	Adrien Giulio Diogo Cristian (from 2527)	Integra-te	External client (Fundação António Aleixo)	<i>Delivered as a separate specification document</i>
B	Pedro Nuno	<i>Pedro's Pitch</i>	Student pitch	Pitch document from <i>Pitching & Entrepreneurship</i> module
C	Gonçalo Nelson	<i>Gonçalo's Pitch</i>	Student pitch	Pitch document from <i>Pitching & Entrepreneurship</i> module
D	Tomás Lucas	<i>Tomás' Pitch</i>	Student pitch	Pitch document from <i>Pitching & Entrepreneurship</i> module

Note on Group A (Integra-te): the full functional, content and client-facing requirements for the Integra-te platform are provided in a **separate briefing document**. Everything in this document (phases, deliverables, evaluation criteria, technical requirements, policies) applies equally to Group A. The separate brief defines *what* is to be built; this document defines *how* it will be delivered, assessed and governed.

Note on the student from Class 2527: this student receives an **adapted individual brief** aligned with the 1st-year curriculum. Their assessment weights are adjusted accordingly and defined in that separate document. They remain a full member of Group A for planning, standups and day-to-day collaboration.

4. Phases & Sessions

The module is structured across **22 sessions of 3 hours each**, organised in **five phases**. The final session is reserved for the in-class presentation and Q&A.

Phase	Focus	Sessions	Deadline
Phase 1	Scope definition & planning	2	Session 2
Phase 2	Design, architecture & data modelling	3	Session 5
Phase 3	Development — MVP	11	Session 16
Phase 4	Final delivery & presentation	1	Session 17

Phase 1 — Scope Definition & Planning (Sessions 1–2)

Deliverables at the end of Phase 1:

- **Scope document:** a side-by-side comparison between the original briefing/pitch and what the group commits to deliver. Explicit out-of-scope items must be listed.
- **Roles & responsibilities matrix:** every member must appear with defined contributions across frontend, backend, architecture, design and documentation.
- **Technical plan:** chosen stack, high-level architecture diagram, containerisation strategy.
- **First interface draft:** low- or mid-fidelity wireframes (Figma or equivalent) covering the main user flows.
- **Presentation outline:** rough skeleton of the final presentation, to be iterated throughout the module.

Phase 2 — Design, Architecture & Data Modelling (Sessions 3–5)

- Finalised UI design (interactive prototype recommended).
- Data model (entity-relationship diagram or equivalent).
- Architecture diagram showing services, communication patterns and data flow.
- Working `docker-compose.yml` skeleton with placeholder services.
- Initial repository structure with README, contribution guide and CI-ready configuration.

Phase 3 — Development: MVP (Sessions 6–16)

- Core user journeys implemented end to end.
- Backend endpoints for the main entities, with basic validation and error handling.
- Frontend pages bound to real backend data (no hard-coded mocks in the main flow).
- Docker Compose running the full stack locally.
- First unit tests written for critical modules.
- Feature completeness against the Phase 1 scope.
- Expanded test suite (coverage usage encouraged); logging; basic observability.

- Polished UI, accessibility checks and responsive behaviour verified.
- Installation and operations documentation finalised.
- **Recorded product demo** (short video) covering the main flows, as a fallback in case of live-demo incidents.
- Final rehearsal of the presentation.

Phase 4 — Final Delivery & Presentation (Sessions 17)

- Version tagged in Git (**v1.0.0** or similar) and pull request merged to the main repository.
 - Final presentation (evaluated separately — see section 8).
 - Live demo **or** recorded demo as agreed with the tutor.
 - Q&A session in which every member must be able to answer questions about any part of the solution.
-

5. Technical Requirements

Requirements are split into **Functional (FR)** and **Non-Functional (NFR)**. All projects must comply with every requirement below unless a documented exception is agreed with the tutor during Phase 1.

5.1 Functional Requirements (FR)

ID	Requirement
FR-01	The solution must implement the user journeys defined in the scope document.
FR-02	The frontend must consume the backend via a clearly defined API (REST or equivalent).
FR-03	The solution must handle at least the basic CRUD operations over the main domain entities.
FR-04	Input validation and meaningful error messages must be present on both client and server sides.
FR-05	The solution must be fully usable without an active internet connection beyond what is strictly required by the domain (the stack must run locally from a fresh clone).

5.2 Non-Functional Requirements (NFR)

ID	Requirement
NFR-01	The entire solution must run via <code>docker compose up</code> from a fresh clone, with no manual setup beyond environment variables documented in <code>.env.example</code> .
NFR-02	Tech stack must be documented and justified. Third-party cloud services (Firebase, Supabase, Vercel, AWS, etc.) could be used as core dependencies. The assumption is single server + Docker + local storage . Any usage of this kind must be justified and properly documented.
NFR-03	The solution must be responsive and usable on desktop, tablet and mobile viewports.
NFR-04	The codebase must be version-controlled in Git, with commits traceable to each member.
NFR-05	Unit tests must be present for critical backend modules. Logging must be implemented for key operations.
NFR-06	Documentation must be kept up to date throughout the project, not written at the end.
NFR-07	Any use of AI coding assistants (Claude, GitHub Copilot, Cursor, etc.) must be declared in the <code>README.md</code> under an "AI Usage" section — see section 10.

5.3 Bonus (not mandatory)

- Use of Kubernetes or Terraform, as introduced in the *Cloud Computing* module.
 - Integration-level testing beyond unit tests.
 - Accessibility beyond baseline (WCAG 2.1 AA).
-

6. Deliverables

By the final delivery date (**26th June 2026, 23:59 WET**), each group must submit:

1. **Source code** — tagged release on GitHub.
2. **README.md** including:
 - Project summary
 - Stack and architecture overview
 - Installation instructions (`docker compose up` and any prerequisites)
 - List of group members and their primary contributions
 - **AI Usage section** (see section 10)

3. **Technical documentation** — architecture diagram, data model, API reference (OpenAPI/Swagger).
 4. **User-facing documentation** — short guide for the end user or backoffice operator.
 5. **Recorded product demo** (Short Video) uploaded to classroom alongside the repository.
 6. **Presentation deck** — to be delivered in the final session.
-

7. Procedures & Git Workflow

Each group follows this workflow:

1. The group **Leader/Owner** (for pitch groups, this is the original pitch author; for Group A, it is the member nominated by the tutor) creates a git repository on Github.
2. Each member clones the fork locally and works on feature branches.
3. Pull requests are opened **within the fork**, reviewed by at least one other group member, and merged to the fork's main branch.
4. Periodically, the Leader creates a **Github release page** reporting changes alongside with a **Git tag**.
5. Commit history must clearly reflect individual contribution — a sample of each member's commits will be reviewed during evaluation.

Leaders are responsible for repository hygiene: branch protection, meaningful commit messages, and a clean commit graph by delivery.

8. Evaluation Criteria

The project is evaluated on a **0–20 point scale** across **seven criteria**. The **oral presentation is evaluated separately** and does not contribute to the project score defined below.

#	Criterion	Points
1	Scope, planning & project management	2
2	Technical implementation (frontend + backend)	5
3	Architecture, containerisation & data model	3
4	UX/UI design & visual identity fidelity	2
5	Documentation & version control hygiene	3
6	Individual contribution & team collaboration	3
7	Responsible & transparent use of AI	2
	Total	20

8.1 Criterion breakdown

1. Scope, planning & project management (2 pts) — quality of the Phase 1 deliverables; realism of the scope; evidence of ongoing planning throughout the module (issues, milestones, standups).

2. Technical implementation (5 pts) — correctness, robustness and quality of the code; completeness of the MVP against the committed scope; presence of unit tests and logging; code readability and maintainability.

3. Architecture, containerisation & data model (3 pts) — coherence of the system architecture; effective use of Docker and Docker Compose; quality of the data model; absence of unjustified third-party/cloud dependencies.

4. UX/UI design & visual identity fidelity (2 pts) — responsiveness; clarity of navigation; fidelity to the client's or pitch's visual identity (colours, typography, hierarchy); accessibility basics.

5. Documentation & version control hygiene (3 pts) — completeness and clarity of README and technical documentation; Git history quality; tagged release; meaningful commit messages.

6. Individual contribution & team collaboration (3 pts) — balanced distribution of technical work across frontend, backend and architecture; evidence of collaboration

(reviews, pairing, feedback loops); each member able to defend work outside their primary area.

7. Responsible & transparent use of AI (2 pts) — explicit declaration of AI tools used; ability to explain and defend any AI-assisted code during the presentation; absence of unverified or unexplained AI-generated content; healthy balance between tool use and individual understanding.

8.2 Individual vs. group score

The final score per student combines:

- The **group score** (section 8, above) — shared across the group.
- An **individual modifier** based on Git contribution analysis, evidence of defending one's work during the final Q&A, and alignment between the role committed to in Phase 1 and what was actually delivered.

A student who is unable to explain portions of the delivered code attributed to them — whether human- or AI-generated — will have their individual score adjusted downward regardless of the group score.

9. Evaluation Methods

Evaluation draws on multiple sources, triangulated rather than treated in isolation:

- **Phase deliverables review** — at the end of each phase, the tutor reviews the artefacts listed in section 4.
 - **Code review** — a sample of commits per member is reviewed for quality and understanding.
 - **Live repository inspection** — CI-ready configuration, issue/PR hygiene, tag on delivery.
 - **Final presentation Q&A** (separate module-level evaluation, but feeds the individual modifier described in §8.2).
 - **Documentation audit** — README, technical docs and AI Usage section.
-

10. Plagiarism & AI Misuse

This course adopts a **transparency-over-prohibition** stance on AI use. AI coding assistants may be used, but the rules below are non-negotiable.

10.1 Allowed

- Using AI tools (Claude, GitHub Copilot, Cursor, etc.) to generate, refactor, explain or review code.
- Using AI tools to draft or polish documentation and commit messages.
- Using AI tools to accelerate learning of a framework, library or concept.

10.2 Required

- **Declare** every AI tool used in a dedicated "AI Usage" section of the [README.md](#), including: tool name, purpose (e.g. "boilerplate generation for FastAPI routers"), and rough scope (e.g. "used across backend controllers, not used in the data layer").
- **Understand** every line of code you submit. If you cannot explain why a block exists, why it was written that way, or how you would modify it, that code should not be in the project until you do.
- **Defend** your work during the presentation. The tutor may ask any member about any portion of the code. "The AI wrote that" is not a valid answer.

10.3 Prohibited

- Submitting AI-generated code, documentation or diagrams **without declaration**.
- Copying code, assets or text from third-party sources (other students' repositories, tutorials, Stack Overflow answers, etc.) without attribution.
- Relying on AI for architectural decisions without being able to justify them.

10.4 Consequences

- **Undeclared AI use** — automatic forfeit of the *Responsible & transparent use of AI* criterion (2 pts) and case-by-case review of the affected technical criteria.
 - **Inability to defend submitted code** — downward adjustment of the individual score proportional to the scope of the unexplained work.
 - **Plagiarism from external sources** — the full project may be invalidated, subject to the school's academic integrity policy.
-

11. Timeline & Key Dates

Milestone	Date
Phase 1 kick-off	Session 1
Phase 1 deliverables due	End of Session 2
Phase 2 deliverables due	End of Session 5
Phase 3 deliverables due	End of Session 16
Final delivery (code, docs, demo, deck)	26th June 2026, 23:59

Late submission policy

- Up to **24 hours late**: -1 point on the group score.
- **24–72 hours late**: -2 points on the group score.
- **More than 72 hours late**: submission is not accepted and the project is graded on what was committed to the reference repository at the deadline.

Exceptions require tutor approval **before** the deadline, not after.

12. Final Considerations

- The brief is a living document. Minor clarifications may be issued during the module; any material change will be communicated in session and versioned in the reference repository.
 - Keep communication tight — with the tutor, between group members, and across groups. A five-minute standup is cheaper than a five-hour debugging session.
 - Documentation written at the end of the project is almost always worse than documentation written alongside the code. Treat it as a deliverable from Phase 1, not from Phase 4.
 - The best projects are not the most ambitious ones — they are the ones whose ambition matches what the team can actually deliver, test, document and defend by 26th June.
 - For anything not covered here, ask the tutor. There is no penalty for asking; there is often one for assuming.
-

Document maintained by the tutor of the Web Programming 2nd-year module, ETIC_Algarve, Academic Year 2025/26.